Text Classification with NLP and Word Embedding's

DATASET SELECTION

The IMDB Movie Reviews dataset is a widely used benchmark for sentiment analysis in Natural Language Processing. It contains 50,000 movie reviews collected from the Internet Movie Database (IMDB), evenly split into 25,000 training and 25,000 test samples. Each review is labeled as either positive or negative, making it a binary classification problem. The dataset is balanced, with an equal number of positive and negative examples, which makes it well-suited for evaluating text classification models using both traditional feature extraction methods (e.g., TF-IDF) and modern word embeddings.

```
Choose files IMDB Dataset.csv
• IMDB Dataset.csv(text/csv) - 66212309 bytes, last modified: 19/10/2019 - 100% done
Saving IMDB Dataset.csv to IMDB Dataset.csv
Shape: (50000, 2)
                                                review sentiment
0  One of the other reviewers has mentioned that ...  positive
1  A wonderful little production. <br /><br />The...  positive
2  I thought this was a wonderful way to spend ti...  positive
3  Basically there's a family where a little boy ...  negative
4  Petter Mattei's "Love in the Time of Money" is...  positive
sentiment
positive    25000
negative    25000
Name: count, dtype: int64
```

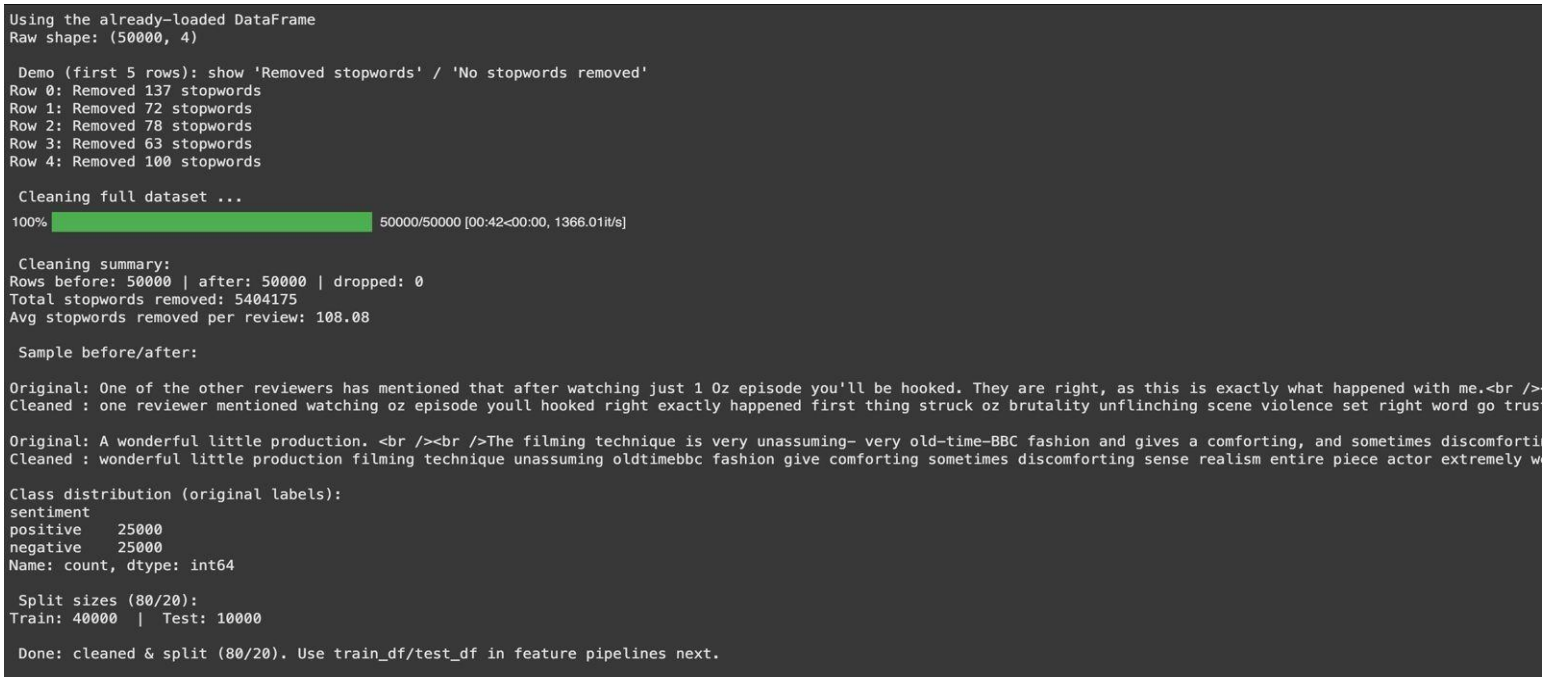
DATA PREPARATION

To build a robust sentiment classification model, the IMDB dataset was cleaned and transformed into a standardized format suitable for feature extraction. The following preprocessing steps were applied:

1. **Lowercasing** – All reviews were converted to lowercase using Python string operations. This avoids treating words like *Movie* and *movie* as separate tokens, reducing redundancy in the vocabulary.
2. **Removal of punctuation, HTML tags, and URLs** – Regular expressions (re.sub) and translation tables (str.maketrans) were used to strip HTML tags (
), hyperlinks, and punctuation symbols. This step helps eliminate noise that does not contribute to sentiment understanding.
3. **Tokenization** – Instead of relying on external tokenizers (which require additional models), a regex-based tokenizer (re.findall(r"[a-z]+", text)) was used to split each review into alphabetic word tokens. This ensures a lightweight, reliable method for breaking text into individual words.

4. **Stopword removal** – A predefined list of English stopwords from NLTK was used to filter out extremely common words (e.g., *the*, *is*, *and*) that do not add meaningful discriminative power for sentiment classification. The process also tracked and reported how many stopwords were removed.
5. **Lemmatization** – Each token was reduced to its base dictionary form using NLTK's WordNetLemmatizer. For example, *running*, *runs*, and *ran* all become *run*. This normalization reduces vocabulary size and groups semantically similar forms under one token.

6. **Stratified Train/Test split (80/20)** – Finally, the cleaned dataset was divided into training (80%) and testing (20%) sets using train_test_split from Scikit-learn with stratification. Stratification ensures the same proportion of positive and negative samples in both sets, preventing data imbalance.



FEATURE EXTRACTION

We converted cleaned reviews into numbers using two complementary representations:

1. TF-IDF to create high-dimensional, sparse vectors that weight words/phrases by how distinctive they are across documents—strong, fast baselines for linear classifiers.

2. Pre-trained embedding's: Using spaCy's en_core_web_md, each token has a 300dimensional pretrained vector. We compute one vector per review by mean-pooling its token vectors (ignoring OOV tokens; empty texts fall back to zeros). These embeddings
   capture semantic similarity and synonymy that TF-IDF can miss, producing X_train_embed/X_test_embed.

We initially attempted gensim Word2Vec/GloVe and Sentence-BERT, but these required large downloads and introduced environment conflicts in Colab (NumPy/Transformers/Torch). Choosing spaCy vectors kept the pipeline stable and lightweight while still meeting the "pretrained embeddings" requirement. Both feature sets are then fed to the same classifiers so we can compare accuracy and robustness across sparse (TF-IDF) vs. dense (embeddings) representations.

```
 Using cleaned text from 'text_clean' in train_df/test_df
 Fitting TF-IDF on training text and transforming train/test ...
TF-IDF ready.
 Train TF-IDF shape: (40000, 10000)
 Test  TF-IDF shape: (10000, 10000)
• Density (train): 0.009438  |  Density (test): 0.009374
• Sample feature names: ['aaron' 'abandon' 'abandoned' 'abbott' 'abc' 'ability' 'able' 'able get'
 'able see' 'aboard' 'abomination' 'abortion' 'abound' 'abraham' 'abrupt'
 'abruptly' 'absence' 'absent' 'absolute' 'absolutely']
```

```
42.8/42.8 MB 22.3 MB/s eta 0:00:00
 Encoding train reviews with spaCy vectors…
100% 40000/40000 [03:50<00:00, 307.61it/s]
 Encoding test reviews with spaCy vectors…
100% 10000/10000 [00:56<00:00, 269.44it/s]
Pretrained embeddings ready.
  • Train spaCy shape: (40000, 300)
  • Test  spaCy shape: (10000, 300)
```

MODEL TRAINING

- Feature representations. We employed two complementary text encodings: (i) TF–IDF with a vocabulary capped at 10,000 n-grams comprising unigrams (single tokens) and bigrams (two-token phrases such as *"not good"*) to capture local context; and (ii) pretrained spaCy embeddings (300-dimensional token vectors) aggregated by mean pooling to obtain one dense vector per review, providing semantic information beyond term frequencies.

- Classifiers. On the TF–IDF features we trained Logistic Regression (strong linear baseline for high-dimensional sparse data) and Multinomial Naïve Bayes (well-suited to non-negative count/weight features). On the embedding features we trained Logistic Regression (preceded by StandardScaler) and a Random Forest (non-linear tree ensemble appropriate for dense, continuous inputs).

- Experimental protocol. Models were trained on a stratified 80/20 train–test split, with random seed = 42 to ensure reproducibility; class weighting was applied where appropriate to mitigate any class imbalance.

- Evaluation metrics. Performance on the held-out test set is reported using Accuracy, Precision (macro), Recall (macro), and F1 (macro), where macro averaging assigns equal weight to each class.

- Practical consideration. For iterative development in Colab, an optional "QUICK" mode subsamples the data and reduces estimator complexity to shorten runtime without altering the evaluation methodology; final results should be computed with the full dataset.

```
 Train: 10000  |  Test: 5000
Training on TF-IDF...

Training on Embeddings...
```

Classifier Comparison (TF-IDF vs. Pretrained Embeddings)

model	accuracy	precision	recall	f1	time_sec
TFIDF + LogisticRegression	0.880800	0.881100	0.880700	0.880700	0.200000
TFIDF + MultinomialNB	0.857000	0.857300	0.856900	0.856900	0.000000
EMBED + LogisticRegression	0.825000	0.825000	0.825000	0.825000	1.400000
EMBED + RandomForest	0.758200	0.758200	0.758200	0.758200	44.500000

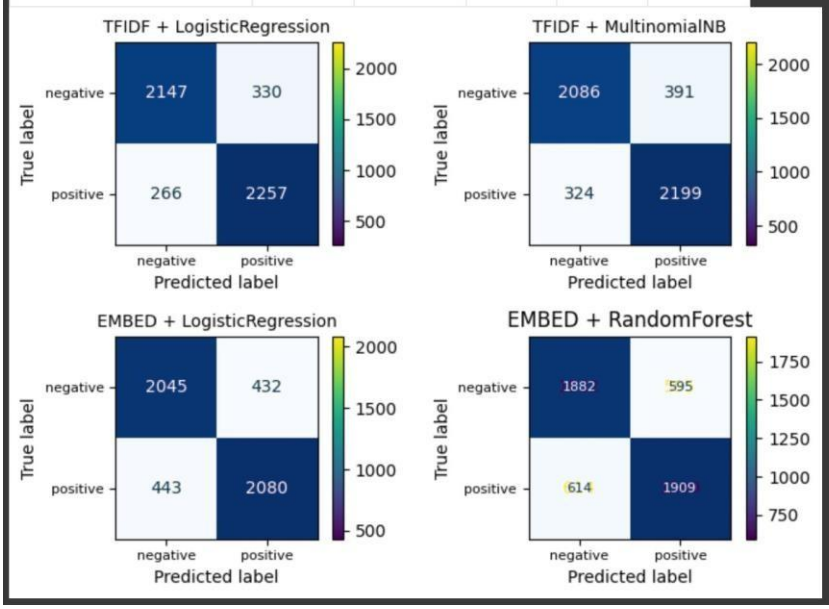
```
LEGENDS
• TF-IDF: Sparse text representation weighting unigrams/bigrams by TF×IDF to emphasize distinctive terms.
• MultinomialNB: Naïve Bayes variant suited to count-like features (TF/TF-IDF); very fast for text.
• Logistic Regression: Strong linear baseline; works well on both sparse TF-IDF and dense embeddings.
• Random Forest: Tree ensemble on dense features (embeddings). Trees=200 (reduced for speed).
• QUICK mode: Train=10000 / Test=5000; toggle QUICK=False for the full dataset.
```

MODEL EVALUATION AND COMPARISON

- Best overall: TF-IDF + Logistic Regression (Acc ≈ 0.881, Macro-F1 ≈ 0.881). Its confusion matrix shows the fewest errors among all models, indicating balanced precision and recall across classes.
- Runner-up: TF-IDF + MultinomialNB (Acc/F1 ≈ 0.857). Strong and fast, but slightly more false positives/negatives than LR.
- Embeddings: spaCy embeddings + Logistic Regression (Acc/F1 ≈ 0.825) trails the TFIDF models; embeddings + Random Forest performs worst (Acc/F1 ≈ 0.758), with noticeably more misclassifications in the confusion matrix.
- Why this pattern: On IMDB sentiment, n-gram TF-IDF captures sentiment cues and negations (e.g., “not good”) very effectively; mean-pooled embeddings dilute polarity signals, and Random Forests are less competitive than linear models here.
- Takeaway: For this task, linear models on TF-IDF remain a very strong baseline. If you want to close the gap with embeddings, use sentence-level pretrained models (e.g., SBERT) or tune LR/feature settings rather than tree ensembles.

Model Comparison (Accuracy / Precision / Recall / F1 — macro)

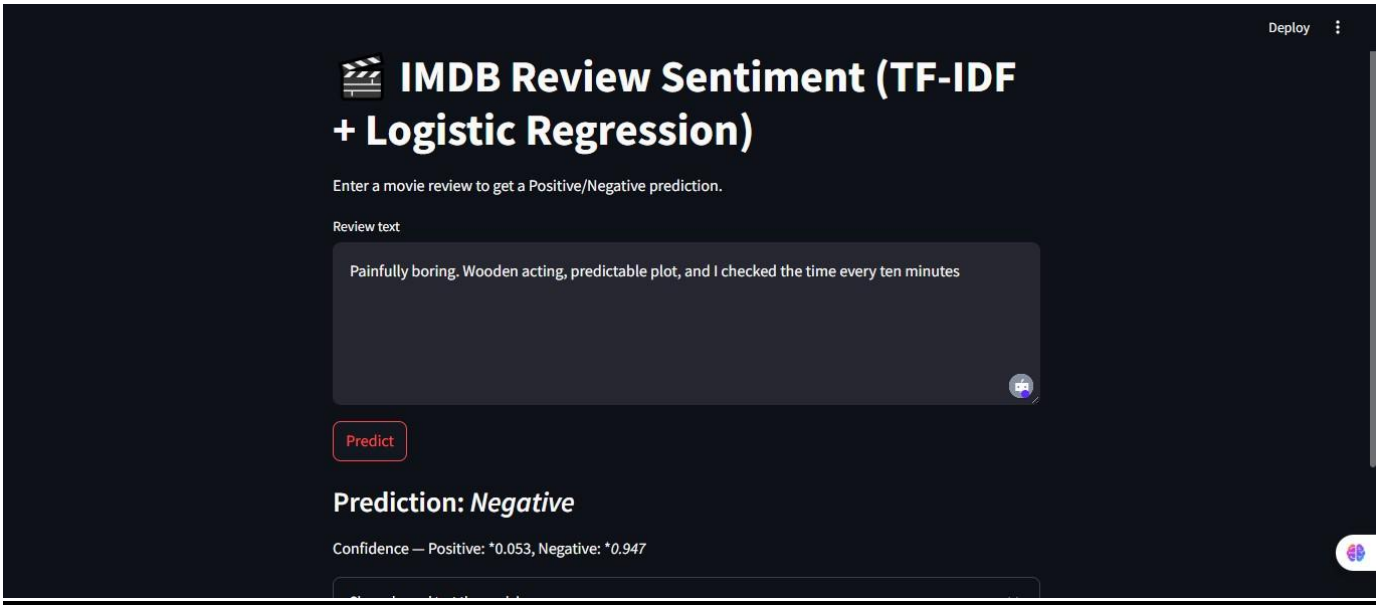
model	accuracy	precision	recall	f1	time_sec
TFIDF + LogisticRegression	0.880800	0.881100	0.880700	0.880700	0.200000
TFIDF + MultinomialNB	0.857000	0.857300	0.856900	0.856900	0.000000
EMBED + LogisticRegression	0.825000	0.825000	0.825000	0.825000	1.400000
EMBED + RandomForest	0.758200	0.758200	0.758200	0.758200	44.500000

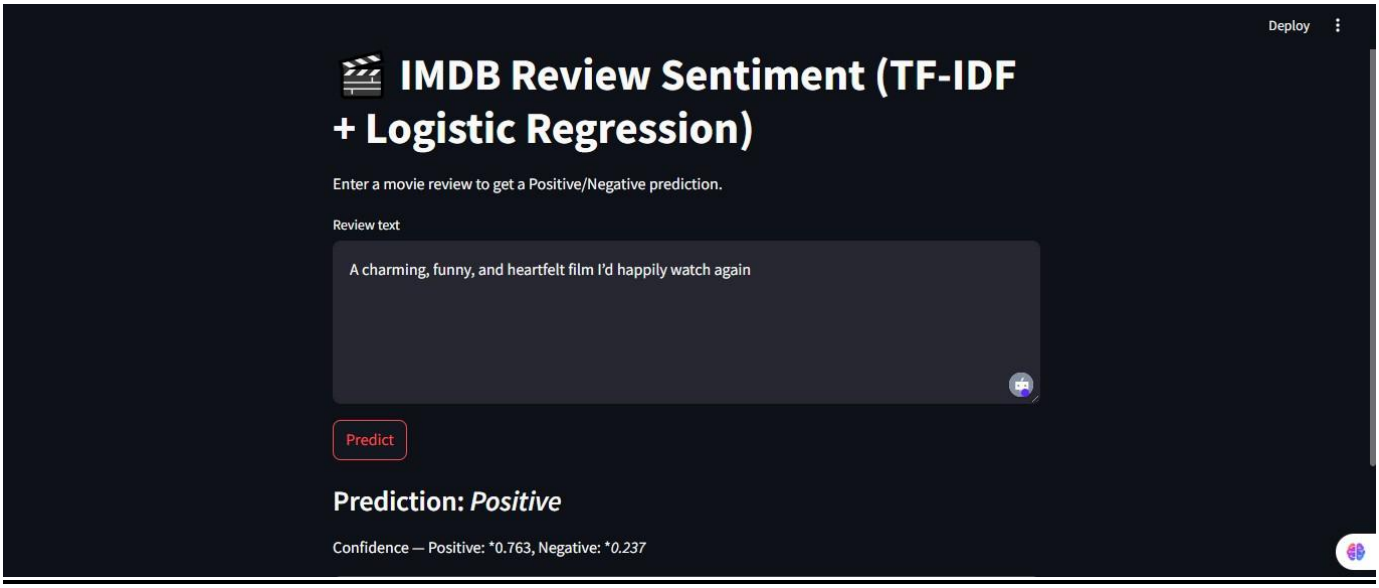


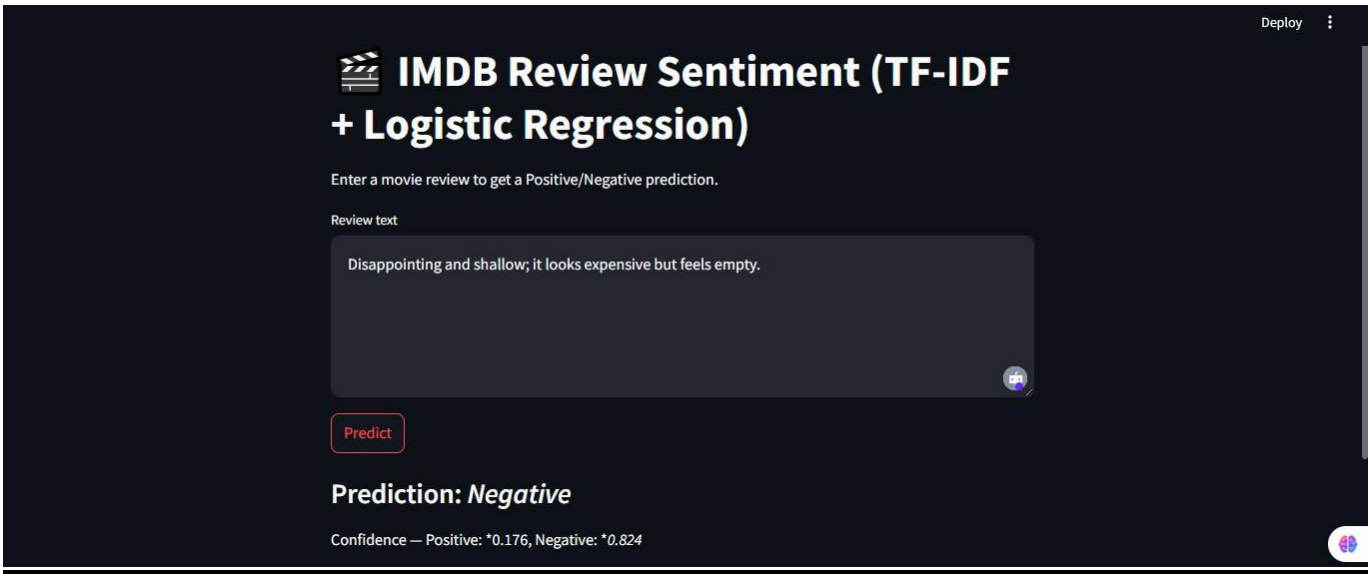

```
TFIDF + LogisticRegression — classification report
              precision    recall  f1-score   support

    negative     0.8898    0.8668    0.8781      2477
    positive     0.8724    0.8946    0.8834      2523

    accuracy                         0.8808      5000
   macro avg     0.8811    0.8807    0.8807      5000
weighted avg     0.8810    0.8808    0.8808      5000


TFIDF + MultinomialNB — classification report
              precision    recall  f1-score   support

    negative     0.8656    0.8421    0.8537      2477
    positive     0.8490    0.8716    0.8602      2523

    accuracy                         0.8570      5000
   macro avg     0.8573    0.8569    0.8569      5000
weighted avg     0.8572    0.8570    0.8570      5000


EMBED + LogisticRegression — classification report
              precision    recall  f1-score   support

    negative     0.8219    0.8256    0.8238      2477
    positive     0.8280    0.8244    0.8262      2523

    accuracy                         0.8250      5000
   macro avg     0.8250    0.8250    0.8250      5000
weighted avg     0.8250    0.8250    0.8250      5000


EMBED + RandomForest — classification report
              precision    recall  f1-score   support

    negative     0.7540    0.7598    0.7569      2477
    positive     0.7624    0.7566    0.7595      2523

    accuracy                         0.7582      5000
   macro avg     0.7582    0.7582    0.7582      5000
weighted avg     0.7582    0.7582    0.7582      5000
```

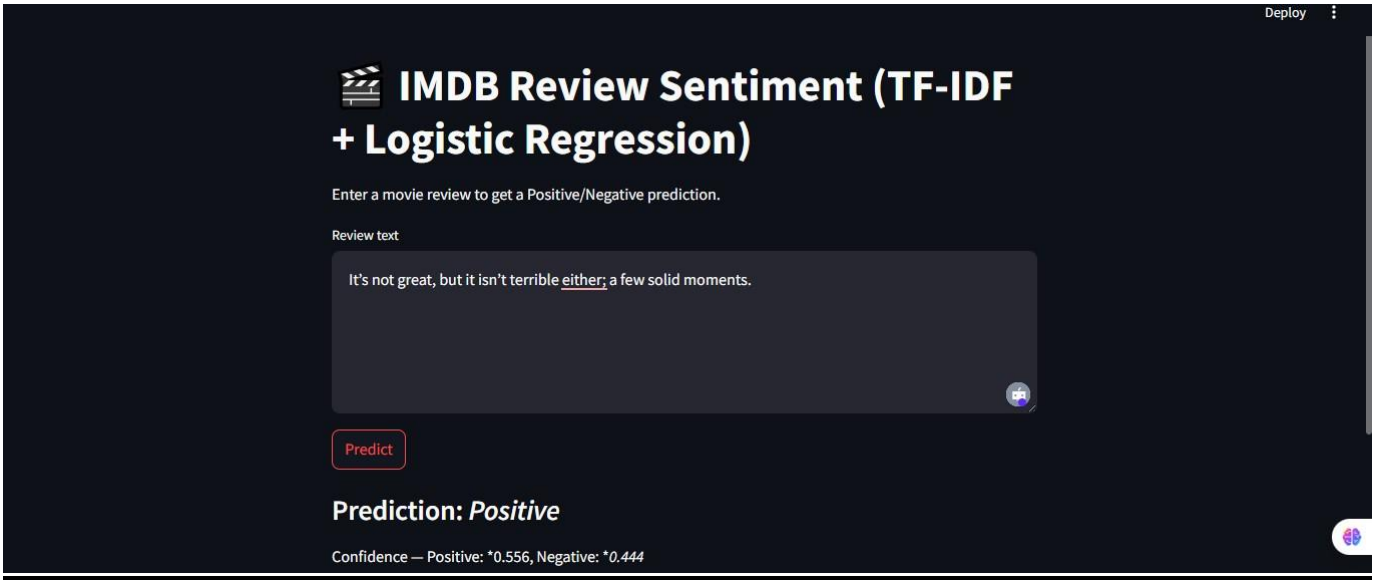
STREAMLIT APP

Typing in various texts and checking predictions:






Deploy
IMDB Review Sentiment (TF-IDF + Logistic Regression)
Enter a movie review to get a Positive/Negative prediction.
Review text
Disappointing and shallow; it looks expensive but feels empty.
Predict
Prediction: Negative
Confidence — Positive: *0.176, Negative: *0.824


Deploy
IMDB Review Sentiment (TF-IDF + Logistic Regression)
Enter a movie review to get a Positive/Negative prediction.
Review text
It's not great, but it isn't terrible either; a few solid moments.
Predict
Prediction: Positive
Confidence — Positive: *0.556, Negative: *0.444